

---

**Algorithm 1** Predicate-aware serializability checking (overview)

---

```
1: procedure CHECKSER( $H$ )
2:   if  $\neg \text{VERIFYINTERNALCONSISTENCY}(H)$  then
3:     return false
4:    $G \leftarrow \text{CREATEKNOWNGRAPH}(H)$ 
5:    $P \leftarrow \text{CREATEPRECEDENCEORACLE}(H)$ 
6:    $C \leftarrow \text{GENERATECONSTRAINTSER}(H, G)$ 
7:   if  $\neg \text{PRUNEWBByREACHABILITY}(G, C, P)$  then
8:     return false
9:    $A \leftarrow \text{CREATEPREDICATEANALYSIS}(G, P)$ 
10:   $\mathcal{R} \leftarrow \text{PREDICATEPRUNING}(H, G, P, A)$ 
11:  if  $\text{HASCONFLICT}(\mathcal{R})$  then
12:    return false
13:   $\mathcal{F} \leftarrow \text{SAT-ENCODE}(H, G, C, P, \mathcal{R})$ 
14:  return  $\text{MONOSAT-SOLVE}(\mathcal{F})$ 
```

---

---

**Algorithm 2** PredicateAnalysis / PredicatePruning.Result

---

```
1: procedure CREATEPREDICATEANALYSIS( $G, P$ )
2:   for all  $\rho = (T, p, i) \in O$  do
3:     if  $(p \in \text{QueryPlan} \wedge p.\text{distinct}) \vee (\neg \text{RowLOCAL}(p) \wedge (p \notin \text{QueryPlan} \vee \neg p.\text{monotone}))$  then
4:       raise QueryException
5:   return  $A = (G, P, \text{BUILDWRITEINDEXES}(G))$ 

6: procedure FINALIZESOURCEDOMAINS( $O, P$ )
7:   for all  $\rho \in \text{dom}(O), k \in O[\rho].K$  do
8:     if  $k.\text{internal} \vee k.a \neq \perp$  then
9:       continue
10:     $k.U \leftarrow \text{PRUNESOURCECANDIDATES}(\rho, k.W, k.Good, P)$ 
11:     $k.\text{badWrites} \leftarrow k.U \setminus k.Good$ 
12:   return  $O$ 

13: procedure PREPARERESIDUALITEMS( $Q, P$ )
14:    $J \leftarrow \emptyset$ 
15:   for all  $q = (T, B, I) \in Q$  with  $\neg q.\text{resolved} \wedge \neg(T \prec_P B)$  do
16:     for all  $I \in I$  do
17:       if  $\exists g \in I : B \prec_P g \wedge g \prec_P T$  then
18:         continue
19:        $I' \leftarrow \{g \in I \setminus \{T, B\} \mid \neg(g \prec_P B \vee T \prec_P g)\}$ 
20:        $\text{outside} \leftarrow q.\text{outside} \wedge \neg(B \prec_P T)$ 
21:        $\text{APPEND}(J, (T, B, \text{outside}, I'))$ 
22:   return  $J$ 

23: procedure BEFOREWRITE( $a, z$ )
24:   if  $\text{tx}(a) = \text{tx}(z)$  then
25:     return  $(a.\text{index} < z.\text{index})$ 
26:   if  $\ell_{\text{tx}(a), \text{tx}(z), a.\text{key}}$  is registered then
27:     return  $\ell_{\text{tx}(a), \text{tx}(z), a.\text{key}}$ 
28:   return  $\text{ORDER}(\text{tx}(a), \text{tx}(z))$ 
```

---

**Algorithm 3** Predicate-aware serializability checking (full version)

---

```

1: procedure CHECKSER( $H$ )
2:   if  $\neg \text{VERIFYINTERNALCONSISTENCY}(H)$  then
3:     return false
4:    $G \leftarrow \text{CREATEKNOWNGRAPH}(H)$ 
5:    $P \leftarrow \text{CREATEPRECEDENCEORACLE}(H)$ 
6:    $C \leftarrow \text{GENERATECONSTRAINTS SER}(H, G)$ 
7:   if  $\neg \text{PRUNEWBYREACHABILITY}(G, C, P)$  then
8:     return false
9:    $A \leftarrow \text{CREATEPREDICATEANALYSIS}(G, P)$ 
10:   $\mathcal{R} \leftarrow \text{PREDICATEPRUNING}(H, G, P, A)$ 
11:  if  $\text{HASCONFLICT}(\mathcal{R})$  then
12:    return false
13:   $\mathcal{F} \leftarrow \text{SAT-ENCODE}(H, G, C, P, \mathcal{R})$ 
14:  return  $\text{MONOSAT-SOLVE}(\mathcal{F})$ 

15: procedure CREATEKNOWNGRAPH( $H$ )
16:   $H \leftarrow \text{ENSUREINITIALVERSIONS}(H)$ 
17:   $V_G \leftarrow \mathcal{T}_H \cup \{T_0\}; E_G \leftarrow \emptyset$ 
18:  for all  $(U, V) \in SO_H$  do
19:     $E_G \leftarrow E_G \cup \{(U, V, \text{SO}, *)\}$ 
20:  for all external point reads  $r(x, v)$  in  $T$  do
21:     $a \leftarrow \text{SOURCEWRITE}(r)$ 
22:     $E_G \leftarrow E_G \cup \{(tx(a), T, \text{WR}, x)\}$ 
23:   $O \leftarrow \text{PREDICATEOBSERVATIONS}(H)$ 
24:  return  $(V_G, E_G, O)$ 

25: procedure CREATEPRECEDENCEORACLE( $H$ )
26:   $P \leftarrow \text{PRECEDENCEORACLE}(\mathcal{T}_H \cup \{T_0\})$ 
27:  return  $P$   $\triangleright$  one shared, initially empty closure

28: procedure GENERATECONSTRAINTS SER( $H, G$ )
29:   $C \leftarrow \emptyset$ 
30:  for all unordered writer pairs  $\{A, B\}$  do
31:     $X \leftarrow \{x \mid A, B \in \text{WriteTx}_x\}$ 
32:    if  $X = \emptyset$  then
33:      continue
34:     $\text{either} \leftarrow \emptyset; \text{or} \leftarrow \emptyset$ 
35:    for all  $x \in X$  do
36:       $\text{either} \leftarrow \text{either} \cup \{(A, B, \text{WW}, x)\}$ 
37:       $\text{or} \leftarrow \text{or} \cup \{(B, A, \text{WW}, x)\}$ 
38:      for all  $A \xrightarrow{\text{WR}(x)} U$  with  $U \neq B$  do
39:         $\text{either} \leftarrow \text{either} \cup \{(U, B, \text{RW}, x)\}$ 
40:      for all  $B \xrightarrow{\text{WR}(x)} U$  with  $U \neq A$  do
41:         $\text{or} \leftarrow \text{or} \cup \{(U, A, \text{RW}, x)\}$ 
42:     $C \leftarrow C \cup \{\langle \text{either}, \text{or} \rangle\}$ 
43:  return  $C$   $\triangleright$  one choice per writer pair

44: procedure PREDICATEPRUNING( $H, G, P, A$ )
45:   $D \leftarrow \text{RECORDEDEXTERNALSOURCEEDGES}(G)$ 
46:   $E_0 \leftarrow \{(T_0, T) \mid T \in \mathcal{T}_H\}$ 
47:   $P \leftarrow \text{TC}(P \cup \text{ends}(E_G \cup D) \cup E_0)$ 
48:  if  $\text{CYCLE}(P)$  then
49:    return  $\mathcal{R}_\perp$ 
50:   $O \leftarrow \text{PREPAREOBSERVATIONS}(H, G, A)$ 
51:  if  $\exists p \in \text{dom}(O) : \neg O[p].\text{valid}$  then
52:    return  $\mathcal{R}_\perp$ 
53:   $Q \leftarrow \text{BUILDGMWR}(O, P)$ 
54:  if  $Q = \perp$  then
55:    return  $\mathcal{R}_\perp$ 
56:  if  $\neg \text{PREPROPAGATEGMWR}(Q, P, D)$  then
57:    return  $\mathcal{R}_\perp$ 
58:   $O \leftarrow \text{FINALIZESOURCEDOMAINS}(O, P)$ 
59:   $J \leftarrow \text{PREPARERESIDUALITEMS}(Q, P)$ 
60:  return  $\mathcal{R} = (A, P, D, O, J)$ 

61: procedure PRUNEWBYREACHABILITY( $G, C, P$ )
62:  if  $C = \emptyset$  then
63:    return true
64:   $E_N \leftarrow \{e \in E_G \mid \text{type}(e) \in \{\text{SO}, \text{WR}, \text{WW}, \text{RW}\}\}$ 
65:   $P \leftarrow \text{TC}(P \cup \text{ends}(E_N))$ 
66:  repeat
67:     $M \leftarrow \emptyset$   $\triangleright$  choices solved in this round
68:    for all  $c = \langle \text{either}, \text{or} \rangle \in C$  do
69:       $c_e \leftarrow \text{WOULDCYCLE}(P, \text{either})$ 
70:       $c_o \leftarrow \text{WOULDCYCLE}(P, \text{or})$ 
71:      if  $c_e \wedge c_o$  then
72:        return false
73:      if  $c_e$  then
74:         $E_G \leftarrow E_G \cup \text{or}$ 
75:         $P \leftarrow \text{TC}(P \cup \text{ends}(\text{or})); M \leftarrow M \cup \{c\}$ 
76:      else if  $c_o$  then
77:         $E_G \leftarrow E_G \cup \text{either}$ 
78:         $P \leftarrow \text{TC}(P \cup \text{ends}(\text{either})); M \leftarrow M \cup \{c\}$ 
79:     $C \leftarrow C \setminus M$ 
80:  until  $C = \emptyset \vee |M| \leq \tau \max(1, |C|)$ 
81:  return true  $\triangleright \tau = 0.01$  in the audited code

82: procedure WOULDCYCLE( $P, B$ )
83:   $B' \leftarrow \{(u, v) \mid (u, v, t, x) \in B, t \in \{\text{WW}, \text{RW}\}\}$ 
84:  return  $\text{CYCLE}(P \cup B')$   $\triangleright$  whole branch; no mutation

85: procedure SAT-ENCODE( $H, G, C, P, \mathcal{R}$ )
Require:  $\mathcal{R}.P \equiv P \wedge \neg \text{HASCONFLICT}(\mathcal{R})$ 
86:   $S \leftarrow (\mathcal{T}_H, \emptyset); BV, CL, E_D \leftarrow \emptyset$ 
87:   $E_K \leftarrow \text{BUILDKNOWNORDER}(G, P)$ 
88:  if  $E_K = \perp$  then
89:    return  $(S, BV, \{\perp\})$ 
90:  for all  $(u, v) \in E_K \cup \text{ends}(\mathcal{R}.D)$  with  $u, v \in \mathcal{T}_H$  do
91:     $CL \leftarrow CL \cup \{\text{EDGE}(S, u, v)\}$ 
92:   $E_D \leftarrow \{(\top, e) \mid e \in E_G \cup \text{Typed}(\mathcal{R}.D)\}$ 
93:  for all  $c = \langle \text{either}, \text{or} \rangle \in C$  do
94:     $b_c \leftarrow \text{FRESHBOOLEAN}; BV \leftarrow BV \cup \{b_c\}$ 
95:    for all  $e \in \text{either}$  do
96:       $E_D \leftarrow E_D \cup \{(b_c, e)\}$ 
97:    for all  $e \in \text{or}$  do
98:       $E_D \leftarrow E_D \cup \{(\neg b_c, e)\}$ 
99:  for all  $(u, v, x)$  represented by a WW entry in  $E_D$  do
100:     $\ell_{u,v,x} \leftarrow \bigvee \{\gamma \mid (\gamma, (u, v, \text{WW}, x)) \in E_D\}$ 
101:     $(CL_P, E_P) \leftarrow \text{ENCODEPREDICATES}(H, G, P, S, \mathcal{R})$ 
102:     $CL \leftarrow CL \cup CL_P; E_D \leftarrow E_D \cup E_P$ 
103:  for all  $(\gamma, (u, v, t, x)) \in E_D$  do
104:     $l \leftarrow \text{ORDER}(u, v)$ 
105:     $CL \leftarrow CL \cup \{\gamma \Rightarrow l\}$ 
106:   $CL \leftarrow CL \cup \{\text{ACYCLIC}(S)\}$ 
107:  return  $\mathcal{F} = (S, BV, CL)$ 

108: procedure ORDER( $u, v$ )
109:  if  $u = v \vee v \prec_P u$  then
110:    return  $\perp$ 
111:  if  $u \prec_P v$  then
112:    return  $\top$ 
113:  if  $\{u, v\}$  has no order literals then
114:     $e_{uv} \leftarrow \text{EDGE}(S, u, v); e_{vu} \leftarrow \text{EDGE}(S, v, u)$ 
115:     $CL \leftarrow CL \cup \{e_{uv} \vee e_{vu}, \neg e_{uv} \vee \neg e_{vu}\}$ 
116:  return  $e_{uv}$   $\triangleright b_{uv}$  abbreviates this call

117: procedure MONOSAT-SOLVE( $\mathcal{F}$ )
118:   $\text{solver} \leftarrow \text{MONOSATSOLVER}(\mathcal{F})$ 
119:  return  $\text{SOLVE}(\text{solver})$ 

```

---

---

**Algorithm 4** Predicate pruning, prepropagation, and complete encoding

---

```
1: procedure BUILDGMWR( $O, P$ )
2:    $Q \leftarrow \emptyset$   $\triangleright$  map keyed by  $(T, B)$ 
3:   for all  $\rho = (T, p, i) \in \text{dom}(O)$  with ROWLOCAL( $p$ ) do
4:     for all  $k \in O[\rho].K$  do
5:       if  $k.\text{internal}$  then
6:         continue
7:       if  $\neg k.\text{valid}$  then
8:         return  $\perp$ 
9:       if  $k.a \neq \perp$  then
10:        continue
11:       $F \leftarrow \text{FRONTIER}(P, k.W, T)$ 
12:      for all  $b \in F \setminus k.\text{Good}$  do
13:         $B \leftarrow \text{tx}(b); I \leftarrow \{\text{tx}(w) \mid w \in k.\text{Good}\} \setminus \{T, B\}$ 
14:         $q \leftarrow \text{GETORCREATE}(Q, (T, B))$ 
15:        APPEND( $q.\text{items}, I$ )  $\triangleright$  do not union repair sets
16:   return  $Q$ 

17: procedure PREPROPAGATEGMWR( $Q, P, D$ )
18:    $Q \leftarrow \text{WORKQUEUE}(Q); J \leftarrow \text{INDEXBYTXN}(Q)$ 
19:   while  $Q \neq \emptyset$  do
20:      $q = (T, B, I) \leftarrow \text{POP}(Q)$ 
21:     if  $q.\text{resolved}$  then
22:       continue
23:     if  $T \prec_P B$  then
24:        $q.\text{resolved} \leftarrow \text{true}; \text{continue}$ 
25:     if  $\forall I \in I, \exists g \in I : B \prec_P g \wedge g \prec_P T$  then
26:        $q.\text{resolved} \leftarrow \text{true}; \text{continue}$ 
27:     if  $B \prec_P T$  then
28:        $q.\text{outside} \leftarrow \text{false}$ 
29:     for all  $I \in I$  do
30:        $I \leftarrow I \setminus \{g \mid g \prec_P B \vee T \prec_P g\}$ 
31:     if  $\exists I \in I : I = \emptyset$  then
32:       if  $\neg q.\text{outside}$  then
33:         return  $\text{false}$ 
34:       if  $\neg \text{PUBLISHORDER}(P, T, B, D, Q, J)$  then
35:         return  $\text{false}$ 
36:       continue
37:     if  $\neg q.\text{outside}$  then
38:       for all  $I \in I$  with  $I = \{g\}$  do
39:         if  $\neg \text{PUBLISHORDER}(P, B, g, D, Q, J)$  then
40:           return  $\text{false}$ 
41:         if  $\neg \text{PUBLISHORDER}(P, g, T, D, Q, J)$  then
42:           return  $\text{false}$ 
43:   return  $\text{true}$ 

44: procedure PUBLISHORDER( $P, u, v, D, Q, J$ )
45:   if  $u = v \vee v \prec_P u$  then
46:     return  $\text{false}$ 
47:    $D \leftarrow D \cup \{\text{ORDERFACT}(u, v)\}$ 
48:   if  $u \prec_P v$  then
49:     return  $\text{true}$ 
50:    $A \leftarrow \{u\} \cup \{a \mid a \prec_P u\}$ 
51:    $B \leftarrow \{v\} \cup \{b \mid v \prec_P b\}$ 
52:    $\Delta \leftarrow (A \times B) \setminus P$ 
53:    $P \leftarrow \text{TC}(P \cup \{(u, v)\})$   $\triangleright$  incremental implementation
54:    $U \leftarrow \{a, b \mid (a, b) \in \Delta\}$ 
55:   for all  $q \in \bigcup_{t \in U} J(t)$  with  $\neg q.\text{resolved}$  do
56:     ENQUEUEIFABSENT( $Q, q$ )
57:   return  $\text{true}$ 

58: procedure FRONTIER( $P, W, T$ )
59:    $F \leftarrow W$ 
60:   for all  $w \in W$  do
61:     if  $T \prec_P \text{tx}(w)$  then
62:        $F \leftarrow F \setminus \{w\}$ 
63:     else if  $\exists z \in W : \text{tx}(w) \prec_P \text{tx}(z) \prec_P T$  then
64:        $F \leftarrow F \setminus \{w\}$ 
65:   return  $F$   $\triangleright$  retain incomparable writers

66: procedure PRUNESOURCECANDIDATES( $\rho, W, \text{Good}, P$ )
67:    $U \leftarrow W; T \leftarrow \rho.\text{txn}$ 
68:   for all  $a \in W \cap \text{Good}$  do
69:     if  $T \prec_P \text{tx}(a)$  then
70:        $U \leftarrow U \setminus \{a\}$ 
71:     else
72:       for all  $z \in W$  with KNOWNWW( $a, z$ ) do
73:         if CHANGED( $\rho.p, a, z$ )  $\wedge \text{tx}(z) \prec_P T$  then
74:            $U \leftarrow U \setminus \{a\}; \text{break}$ 
75:   return FRONTIER( $P, U, T$ )

76: procedure LATESTGUARDS( $T, W$ )
77:   for all  $w \in W$  do
78:      $v_w \leftarrow b_{\text{tx}(w), T}$ 
79:   for all  $w \in W$  do
80:      $\lambda_w \leftarrow v_w \wedge \bigwedge_{z \in W \setminus \{w\}} \neg(\text{BEFOREWRITE}(w, z) \wedge v_z)$ 
81:   return  $\{(w, \lambda_w) \mid w \in W\}$ 

82: procedure ROWDEPENDENCIES( $\rho, x, a, \lambda_a, W, U$ )
83:    $T \leftarrow \rho.\text{txn}; E \leftarrow \{(\lambda_a, (\text{tx}(a), T, \text{PR\_WR}, x))\}$ 
84:   for all  $z \in W \setminus \{a\}$  do
85:     if  $\text{tx}(z) \prec_P T \wedge z \notin U$  then
86:       continue
87:     if CHANGED( $\rho.p, a, z$ ) then
88:        $\gamma \leftarrow \lambda_a \wedge \text{BEFOREWRITE}(a, z)$ 
89:        $E \leftarrow E \cup \{(\gamma, (T, \text{tx}(z), \text{PR\_RW}, x))\}$ 
90:   return  $E$ 
```

---

**Algorithm 4** Predicate pruning, prepropagation, and complete encoding (continued)

---

```

91: procedure ENCODEPREDICATES( $H, G, P, S, \mathcal{R}$ )
92:    $CL_P, E_P \leftarrow \emptyset$ 
93:   for all  $\rho = (T, p, i) \in O$  do
94:      $o \leftarrow \mathcal{R}.O[\rho]$ ;  $Src \leftarrow o.Src$ 
95:     if  $\neg o.valid$  then
96:        $CL_P \leftarrow CL_P \cup \{\perp\}$ ; continue
97:     if  $\neg \text{RowLocal}(p)$  then
98:        $(C', E') \leftarrow \text{ENCODEQUERYPLAN}(\rho, Src)$ 
99:        $CL_P \leftarrow CL_P \cup C'$ ;  $E_P \leftarrow E_P \cup E'$ 
100:    continue
101:    for all  $k \in o.K$  do
102:      if  $\neg k.valid$  then
103:         $CL_P \leftarrow CL_P \cup \{\perp\}$ ; continue
104:      if  $k.internal$  then
105:        continue
106:      if  $k.a \neq \perp$  then
107:         $(C', E') \leftarrow \text{ENCODEFIXEDSOURCE}(\rho, k.x, k.a, k.W)$ 
108:      else
109:         $(C', E') \leftarrow \text{ENCODEABSENTKEY}(\rho, k)$ 
110:         $CL_P \leftarrow CL_P \cup C'$ ;  $E_P \leftarrow E_P \cup E'$ 
111:     $CL_P \leftarrow CL_P \cup \text{ENCODERESIDUALGMWR}(\mathcal{R}.J)$ 
112:     $E_P \leftarrow \text{COALESCEPREDICATEDEPS}(E_P)$ 
113:    return  $(CL_P, E_P)$ 

114: procedure FIXEDLATESTCLAUSES( $T, a, W$ )
115:   if  $a \notin W$  then
116:     return  $\{\perp\}$ 
117:    $C \leftarrow \{b_{\text{tx}(a), T}\}$ 
118:   for all  $z \in W \setminus \{a\}$  do
119:      $C \leftarrow C \cup \{\neg \text{BEFOREWRITE}(a, z) \vee \neg b_{\text{tx}(z), T}\}$ 
120:   return  $C$ 

121: procedure ENCODEFIXEDSOURCE( $\rho, x, a, W$ )
122:    $C \leftarrow \text{FIXEDLATESTCLAUSES}(\rho.txn, a, W)$ 
123:   if  $C = \{\perp\}$  then
124:     return  $(C, \emptyset)$ 
125:    $E \leftarrow \text{ROWDEPENDENCIES}(\rho, x, a, \top, W, W)$ 
126:   return  $(C, E)$ 

127: procedure ENCODEABSENTKEY( $\rho, k$ )
128:    $T \leftarrow \rho.txn$ ;  $C, E \leftarrow \emptyset$ 
129:    $(x, W, Good, U) \leftarrow (k.x, k.W, k.Good, k.U)$ 
130:   if  $U = \{a\} \wedge \text{tx}(a) \prec_P T$  then
131:     if  $a \notin Good$  then
132:       return  $(\{\perp\}, \emptyset)$ 
133:      $L \leftarrow \{(a, \top)\}$ 
134:   else
135:      $L \leftarrow \text{LATESTGUARDS}(T, U)$ 
136:   for all  $(a, \lambda_a) \in L$  with  $a \in Good$  do
137:      $E \leftarrow E \cup \text{ROWDEPENDENCIES}(\rho, x, a, \lambda_a, W, U)$ 
138:   return  $(C, E)$   $\triangleright$  result validity comes from GMWR

139: procedure ENCODERESIDUALGMWR( $J$ )
140:    $C \leftarrow \emptyset$ 
141:   for all  $(T, B, outside, I) \in J$  do
142:      $L \leftarrow \emptyset$ 
143:     if  $outside$  then
144:        $L \leftarrow L \cup \{b_{T, B}\}$ 
145:     for all  $g \in I$  do
146:        $L \leftarrow L \cup \{b_{B, g} \wedge b_{g, T}\}$ 
147:     if  $\top \notin L$  then
148:        $C \leftarrow C \cup \{\bigvee (L \setminus \{\perp\})\}$ 
149:   return  $C$   $\triangleright \bigvee \emptyset = \perp$ ; items are conjunctive

150: procedure ENCODEQUERYPLAN( $\rho, Src$ )
151:    $T \leftarrow \rho.txn$ ;  $C, E, V \leftarrow \emptyset$ 
152:   for all  $x \in \text{SCOPEDKEYS}(\rho.p)$  do
153:      $W_x \leftarrow \text{LATESTEXT}(x, T)$ 
154:      $a \leftarrow \text{LATESTSELFBETWEEN}(x, T, \rho.index)$ 
155:     if  $a \neq \perp$  then
156:       if  $Src(x) \neq \perp \wedge Src(x) \neq a$  then
157:         return  $(\{\perp\}, \emptyset)$ 
158:        $L_x \leftarrow \{(a, \top)\}$ 
159:        $\text{ADDVERSIONS}(V, \{a\}, L_x)$ 
160:     else if  $Src(x) = a \neq \perp$  then
161:        $C_a \leftarrow \text{FIXEDLATESTCLAUSES}(T, a, W_x)$ 
162:        $C \leftarrow C \cup C_a$ ;  $L_x \leftarrow \{(a, \top)\}$ 
163:        $\text{ADDVERSIONS}(V, W_x, L_x)$ 
164:     else
165:        $L_x \leftarrow \text{LATESTGUARDS}(T, W_x)$ 
166:        $\text{ADDVERSIONS}(V, W_x, L_x)$ 
167:     for all  $(a, \lambda_a) \in L_x$  with  $\text{tx}(a) \neq T$  do
168:        $E \leftarrow E \cup \{(\lambda_a, (\text{tx}(a), T, \text{PR\_WR}, x))\}$ 
169:    $\mathcal{B} \leftarrow \text{CONTRIBUTINGBINDINGS}(\rho.p, V)$ 
170:    $\kappa \leftarrow (w \mapsto \perp)$   $\triangleright$  default context for every write
171:   for all  $\beta \in \mathcal{B}$  do
172:     if  $\beta \not\subseteq Src$  and every  $w \in \beta$  is selectable then
173:        $C \leftarrow C \cup \{\bigvee_{w \in \beta} \neg \lambda_w\}$ 
174:     for all  $w \in \beta$  do
175:       if every  $z \in \beta \setminus \{w\}$  is selectable then
176:          $\kappa_w \leftarrow \kappa_w \vee \bigwedge_{z \in \beta \setminus \{w\}} \lambda_z$ 
177:   for all external frontiers  $L_x$  do
178:     for all  $(a, \lambda_a) \in L_x, z \in W_x \setminus \{a\}$  do
179:        $\gamma \leftarrow \lambda_a \wedge \text{BEFOREWRITE}(a, z) \wedge (\kappa_a \vee \kappa_z)$ 
180:        $E \leftarrow E \cup \{(\gamma, (T, \text{tx}(z), \text{PR\_RW}, x))\}$ 
181:   return  $(C, E)$   $\triangleright$  complete before solving

182: procedure COALESCEPREDICATEDEPS( $E$ )
183:    $M, E' \leftarrow \emptyset$ 
184:   for all  $(\gamma, (u, v, t, x)) \in E$  do
185:     if  $(u, v, t) \notin \text{dom}(M)$  then
186:        $M[u, v, t] \leftarrow (u, v, t, \emptyset)$ 
187:      $e \leftarrow M[u, v, t]$ ;  $e.keys \leftarrow e.keys \cup \{x\}$ 
188:      $E' \leftarrow E' \cup \{(\gamma, e)\}$ 
189:   return  $E'$   $\triangleright$  each support still implies its edge

```

---